

Graph Fundamentals

1 Introduction to Graphs

A graph is a data structure used to represent relationships or connections between different objects.

A graph consists of:

- **Vertices** – the objects or points.
- **Edges** – the connections between the objects.

A graph is represented mathematically as:

$$G = (V, E)$$

where:

- V = set of vertices
- E = set of edges

For example, a graph representing four cities and the roads between them can be represented as:

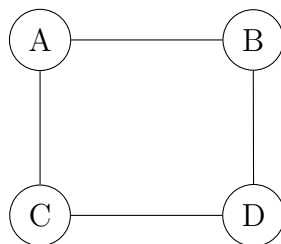
$$V = \{A, B, C, D\}$$

and

$$E = \{(A, B), (A, C), (B, D), (C, D)\}$$

2 Simple Graph Example

Consider the following graph:



Here:

$$V = \{A, B, C, D\}$$

and

$$E = \{(A, B), (A, C), (B, D), (C, D)\}$$

There are:

$$|V| = 4$$

vertices and

$$|E| = 4$$

edges.

3 Basic Terminology

3.1 Vertex

A vertex is a point or object in a graph.

It is also called a:

- node
- vertex

For example:

$$A, B, C, D$$

are vertices.

3.2 Edge

An edge represents a connection between two vertices.

For example:

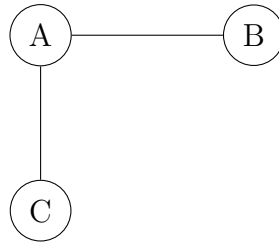
$$(A, B)$$

represents an edge between A and B .

3.3 Adjacent Vertices

Two vertices are called **adjacent** if there is an edge directly connecting them.

In the following graph:



A is adjacent to B and C .

But B and C are not adjacent because there is no direct edge between them.

3.4 Incident Edge

An edge is said to be **incident** on a vertex if the edge is connected to that vertex.

For example, if:

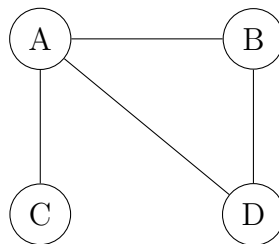
$$E = (A, B)$$

then edge E is incident on both A and B .

3.5 Degree of a Vertex

The degree of a vertex is the number of edges connected to that vertex.

For example:



For vertex A :

$$\text{degree}(A) = 3$$

because three edges are connected to A .

For vertex B :

$$\text{degree}(B) = 2$$

4 Path

A path is a sequence of vertices where consecutive vertices are connected by edges.

For example:

$$A \rightarrow B \rightarrow D$$

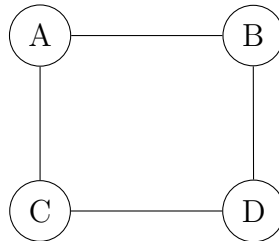
is a path if the graph contains:

(A, B)

and

 (B, D)

Consider:

Possible paths from A to D are: $A \rightarrow B \rightarrow D$

and

 $A \rightarrow C \rightarrow D$

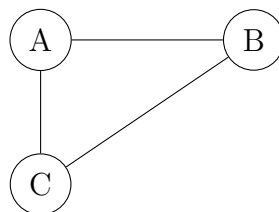
5 Cycle

A cycle is a path that starts and ends at the same vertex.

For example:

 $A \rightarrow B \rightarrow C \rightarrow A$

forms a cycle.



The cycle is:

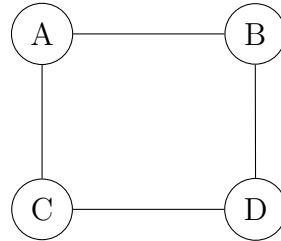
 $A \rightarrow B \rightarrow C \rightarrow A$ Therefore, graphs **can contain cycles**.

This is an important difference between general graphs and trees.

6 Connected Graph

A graph is connected if there is a path between every pair of vertices.

Example:



There is a path between every pair of vertices.

Therefore, the graph is connected.

7 Disconnected Graph

A graph is disconnected if at least one vertex or group of vertices cannot be reached from another group.



There is no path from A to C .

Therefore, this graph is disconnected.

8 Undirected Graph

In an undirected graph, an edge does not have a direction.

For example:

$$(A, B)$$

means that A is connected to B and B is connected to A .



There is no arrow because the edge has no direction.

9 Directed Graph

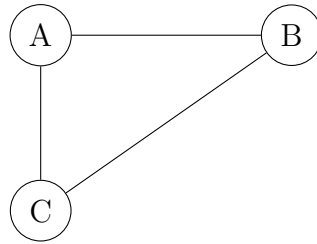
In a directed graph, every edge has a direction.

For example:

$$A \rightarrow B$$

means that there is an edge from A to B .

It does not necessarily mean that there is an edge from B to A .



For example:

$$A \rightarrow B$$

and

$$B \rightarrow C$$

are directed edges.

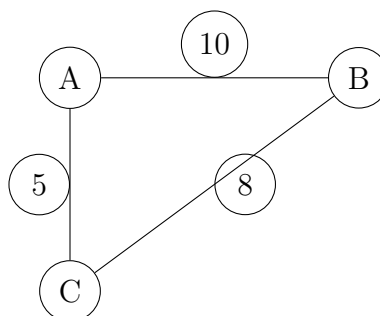
10 Weighted Graph

A weighted graph assigns a value or weight to each edge.

The weight can represent:

- distance
- cost
- time
- capacity
- price

Example:



The edge from A to B has weight 10.

The edge from A to C has weight 5.

11 Self Loop

A self-loop is an edge that starts and ends at the same vertex.

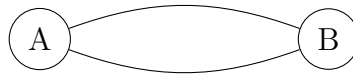


The edge is:

(A, A)

12 Multiple Edges

A graph may allow more than one edge between the same pair of vertices.



Such edges are called **parallel or multiple edges**.

13 Graph Representations

A graph can be represented mainly using:

1. Adjacency Matrix
2. Adjacency List
3. Edge List

14 Adjacency Matrix

An adjacency matrix is a two-dimensional array used to represent connections between vertices.

For a graph having n vertices, the adjacency matrix is:

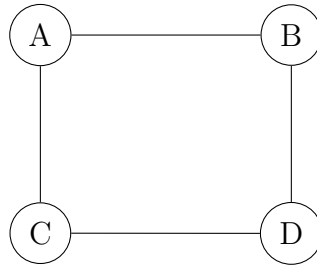
$$n \times n$$

For an unweighted graph:

$$A[i][j] = \begin{cases} 1, & \text{if an edge exists from } i \text{ to } j \\ 0, & \text{otherwise} \end{cases}$$

14.1 Example 1: Simple Undirected Graph

Consider:



Vertices:

$$V = \{A, B, C, D\}$$

Edges:

$$E = \{AB, AC, BD, CD\}$$

The adjacency matrix is:

	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>
<i>A</i>	0	1	1	0
<i>B</i>	1	0	0	1
<i>C</i>	1	0	0	1
<i>D</i>	0	1	1	0

For example:

$$A[A][B] = 1$$

because there is an edge between *A* and *B*.

But:

$$A[A][D] = 0$$

because there is no direct edge between *A* and *D*.

14.2 Why is the Matrix Symmetric?

For an undirected graph:

$$A[i][j] = A[j][i]$$

For example:

$$A[A][B] = 1$$

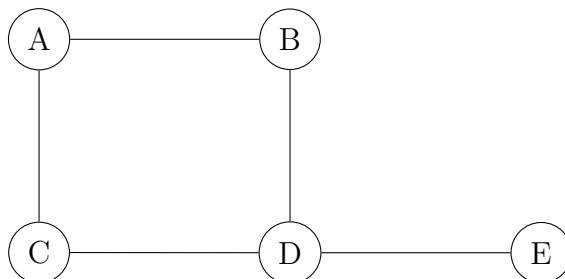
and:

$$A[B][A] = 1$$

Therefore, the adjacency matrix of an undirected graph is **symmetric**.

15 Example 2: Another Graph

Consider:



The adjacency matrix is:

	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>
<i>A</i>	0	1	1	0	0
<i>B</i>	1	0	0	1	0
<i>C</i>	1	0	0	1	0
<i>D</i>	0	1	1	0	1
<i>E</i>	0	0	0	1	0

Notice that row *D* contains three 1s.

Therefore:

$$\text{degree}(D) = 3$$

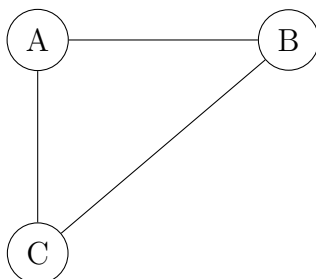
This gives an important observation:

Number of 1s in a row = degree of that vertex

for a simple undirected graph.

16 Example 3: Directed Graph

Consider:



The edges are:

$$A \rightarrow B$$

$$B \rightarrow C$$

$$C \rightarrow A$$

The adjacency matrix is:

	<i>A</i>	<i>B</i>	<i>C</i>
<i>A</i>	0	1	0
<i>B</i>	0	0	1
<i>C</i>	1	0	0

Notice:

$$A[A][B] = 1$$

but:

$$A[B][A] = 0$$

because the edge exists only from *A* to *B*.

Therefore, a directed graph's adjacency matrix need not be symmetric.

17 Adjacency List

An adjacency list stores the neighboring vertices of each vertex.

For Example 1:

Vertex	Adjacent Vertices
<i>A</i>	<i>B, C</i>
<i>B</i>	<i>A, D</i>
<i>C</i>	<i>A, D</i>
<i>D</i>	<i>B, C</i>

The same graph can therefore be represented as:

$$A \rightarrow B, C$$

$$B \rightarrow A, D$$

$$C \rightarrow A, D$$

$$D \rightarrow B, C$$

18 Adjacency Matrix vs Adjacency List

	Adjacency Matrix	Adjacency List
Storage	$O(V^2)$	$O(V + E)$
Check edge	$O(1)$	$O(\text{degree})$
Good for	Dense graphs	Sparse graphs
Representation	2-D array	List of neighbors

A graph is called **dense** when it has many edges.

A graph is called **sparse** when it has relatively few edges.

19 Graph vs Tree

A tree is a special type of graph.

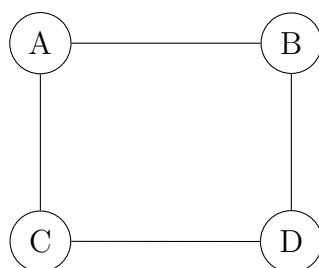
However, a general graph does not have all the restrictions of a tree.

Graph	Tree
Can have cycles	Cannot contain cycles
No parent-child relationship is required	Usually represented using parent-child relationship
Can have multiple paths between two vertices	Exactly one simple path between any two vertices
Can be disconnected	Always connected
Can be directed or undirected	Usually treated as an undirected connected acyclic graph in basic graph theory
Number of edges can vary	A tree with n vertices has exactly $n - 1$ edges
No root is required	A rooted tree has a root

20 Important Difference: Parent and Child

One of the most important differences is that a general graph does **not require a parent-child relationship**.

For example:



Here, there is no requirement that:

$$A = \text{parent of } B$$

or:

$$B = \text{parent of } D$$

The vertices are simply connected by edges.

In a tree, especially a rooted tree, we normally discuss:

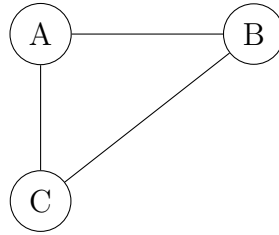
$$\text{Parent} \rightarrow \text{Child}$$

relationships.

21 Important Difference: Cycles

A tree cannot contain a cycle.

For example:



This graph contains:

$$A \rightarrow B \rightarrow C \rightarrow A$$

Therefore, it contains a cycle.

It is a graph, but it is **not a tree**.

22 Important Difference: Number of Edges

For a tree containing n vertices:

$$E = n - 1$$

For example, if:

$$n = 5$$

then a tree must contain:

$$E = 5 - 1 = 4$$

edges.

A general graph does not have to contain exactly $n - 1$ edges.

It can have fewer or more edges, depending on the type of graph.

23 Tree as a Special Graph

A useful way to remember the relationship is:

$$\boxed{\text{Every tree is a graph}}$$

but:

$$\boxed{\text{Every graph is not a tree}}$$

A tree can be considered a graph with additional restrictions:

- connected
- no cycles
- exactly $n - 1$ edges for n vertices

24 Graph Applications

Graphs are widely used to represent real-world relationships.

Application	Graph Representation
Google Maps	Cities and roads
Social networks	People and friendships
Computer networks	Computers and connections
Airline networks	Airports and flights
Web pages	Pages and hyperlinks
Project scheduling	Tasks and dependencies
Recommendation systems	Users and items
Internet routing	Routers and communication links

25 Graph Terminology – Quick Revision

Term	Meaning
Vertex	Point/node in a graph
Edge	Connection between two vertices
Adjacent vertices	Vertices directly connected by an edge
Degree	Number of edges connected to a vertex
Path	Sequence of connected vertices
Cycle	Path that returns to its starting vertex
Connected graph	Path exists between every pair of vertices
Disconnected graph	Some vertices cannot reach others
Undirected graph	Edges have no direction
Directed graph	Edges have direction
Weighted graph	Edges have associated values
Self-loop	Edge from a vertex to itself
Adjacency matrix	$V \times V$ matrix representing connections
Adjacency list	List of neighboring vertices

26 Key Points

1. A graph is represented as:

$$G = (V, E)$$

2. V represents vertices.
3. E represents edges.
4. Graphs can contain cycles.
5. General graphs do not require parent-child relationships.
6. Graphs can be directed or undirected.

7. Graphs can be weighted or unweighted.
8. An adjacency matrix uses a $V \times V$ matrix.
9. In an undirected graph:

$$A[i][j] = A[j][i]$$

10. A tree is a special type of graph.
11. A tree is connected and contains no cycles.
12. A tree with n vertices contains:

$$n - 1$$

edges.

27 One-Line Difference Between Graph and Tree

Graph: vertices connected by edges; cycles may exist and no parent-child relationship is
--

Tree: connected, acyclic graph with a unique path between every pair of vertices.
